



Microwave Timer System Controlled by a PIC18F45K22

CPEN305 - Microcontrollers Lab

Prepared by:

Mario Makhoul (A2220007)

Nancy Dahdah (A2211690)

Rebecca Chalouhi (A2310120)

Instructor: Dr. Katia Karam

December 13, 2025

Contents

1. Introduction	3
2. Background.....	3
3. Objective.....	4
4. Code and Calculations	4
4.1. Code	4
4.2. Explanation Code.....	14
4.3. Calculations	19
5. Proteus.....	20
6. Hardware.....	23
6.1. Components	23
6.2. Datasheets.....	24
6.3. Power Supply and Voltage Regulator	26
6.4. Microcontroller Wiring	27
6.5. 7-Segment Display Wiring	27
6.6. Programming with PICKit (Steps).....	28
6.7. Screenshots	35
7. Future Work	36
8. Conclusion.....	36

Figures

Figure 1: Proteus MinValue.....	21
Figure 2: Proteus MidValue.....	21
Figure 3: Proteus MaxValue	22
Figure 4: Proteus Running	22
Figure 5: Proteus Running Buzzer.....	23
Figure 6: PIC18F45K22 Datasheet.....	24
Figure 7: Servo Motor Datasheet	25
Figure 8: Voltage Regulator Datasheet	25
Figure 9: 7-Segment CA Datasheet.....	26
Figure 10 Circuitry of 7 segment display.....	28
Figure 11 hardware-1	29
Figure 12 hardware-2	30
Figure 13 hardware-3	31

Figure 14 hardware-4	32
Figure 15 hardware-5	33
Figure 16 hardware-6	34
Figure 17 hardware-7	34
Figure 18 final hardware.....	35

1. Introduction

This project implements a small “microwave-style” timer system using the **PIC18F45K22** microcontroller.

The user chooses a time value with a potentiometer, the value is shown on a **2-digit common-anode 7-segment display**, and a **countdown** starts when the push button is pressed and released.

During the countdown:

- An **LED** turns ON.
- An **FS5103R continuous-rotation servo motor** is driven with a simple PWM signal.
- A **buzzer** on RA4 beeps at the last **3, 2 and 1 seconds** to simulate the microwave “alarm”.

The complete system was first tested on **Proteus** and then prepared for **real hardware implementation**.

2. Background

Many embedded applications combine several microcontroller features in one project: analogue-to-digital conversion, timer interrupts, multiplexed displays, debouncing and simple actuators (LEDs, buzzers, motors).

A typical example is a **microwave oven controller**, where the user sets a time, presses Start, and receives visual and audio feedback when the timer finishes.

The **PIC18F45K22** is suitable for this task because it provides:

- 10-bit ADC channels (AN0 used for the potentiometer),
- Several digital I/O ports for the 7-segment display and control signals,
- **Timer0** with interrupt capability,
- Enough program memory for structured C code.

PWM (Pulse Width Modulation) is especially important in this project since it is used to control the continuous-rotation servo. PWM works by switching a digital output pin ON and OFF rapidly, and the ratio between the ON-time and OFF-time determines the output effect. For servos operating at ~50 Hz, a pulse of around 1.5 ms holds the servo at neutral, while longer

pulses (≈ 2 ms) cause rotation. This control technique is widely used in embedded systems due to its simplicity and low power consumption.

The project follows the style of the **CPEN305 lab experiments previously done**, especially:

- ADC configuration on AN0,
- Handling switches and debouncing,
- Using Timer0 as a time base,
- Driving peripherals like LEDs and buzzers.

3. Objective

- Reads a **0–99 s** time value from a **potentiometer** on AN0.
- Displays the selected time on a **2-digit common-anode 7-segment display**.
- Starts the countdown **only after** a clean push button **press and release** (debounced).
- Keeps an **LED** and **servo motor** active while time is running.
- Triggers a **buzzer** during the last **3 seconds** (3, 2, 1 s remaining).
- Uses **Timer0 interrupts** for display multiplexing and for building an approximate **1-second tick**.

4. Code and Calculations

4.1. Code

```
// PIC18F45K22 timer project
// 2-digit CA 7-segment on PORTD + RC0/RC1
// RA0 : pot -> pick a value 00..99
// RA1 : push button -> start after press + release
// RA2 : LED on while counting
// RA3 : servo (simple PWM, around 50 Hz)
// RA4 : buzzer beeps at 3, 2, 1 sec

//Libraries:
```

```

#include <p18cxxx.h>
#include <BCDlib.h>
#include <delays.h>

// I/O shortcuts

#define BTN          PORTAbits.RA1    // push button (active high)
#define LED          PORTAbits.RA2    // LED
#define ServoPWM     PORTAbits.RA3    // servo signal
#define Buzzer       PORTAbits.RA4    // buzzer

// 7-segment (common anode)
// segments on PORTD, digit select on RC0/RC1
#define DIGIT1       PORTCbits.RC0    // ones (right digit)
#define DIGIT2       PORTCbits.RC1    // tens  (left digit)

// ----- global variables -----

unsigned char qstates = 0x01;          // which digit is enabled
now

unsigned char mux_idx = 0;              // 0 = ones, 1 = tens

unsigned char bcd[3] = {0};            // Bin2Bcd buffer [hundreds,
tens, ones]

unsigned char adc_raw  = 0;             // raw ADC from ADRESH
(0..255)

unsigned char set_val  = 0;             // selected value from pot
(0..99)

unsigned char counting = 0;             // 0 = idle, 1 = countdown
started

unsigned char seconds  = 0;             // remaining time

```

```

unsigned char t0_ticks  = 250;          // 250 * 4ms ? 1 second
unsigned char buzz_flag = 0;           // set by ISR when we need a
beep

// 7-segment table for common anode (0 = LED ON)
const unsigned char SS_CA[10] = {
    0xC0, // 0
    0xF9, // 1
    0xA4, // 2
    0xB0, // 3
    0x99, // 4
    0x92, // 5
    0x82, // 6
    0xF8, // 7
    0x80, // 8
    0x90  // 9
};

// prototypes
static void Setup(void);
static void ReadPot(void);
static void CheckButton(void);
static void ServoTask(void);
static void Beep(unsigned int LoopCount);

// ===== main =====
void main(void)
{

```

```

Setup();

while(1)
{
    if (!counting)
    {
        // still choosing the value
        ReadPot();
        CheckButton();        // only reacts when not counting
    }
    else
    {
        // countdown running
        ServoTask();

        if (buzz_flag)
        {
            Beep(200);        // small beep
            buzz_flag = 0;
        }
    }
}

// ===== setup =====
static void Setup(void)
{
    // RC0 / RC1 -> digit select
    TRISCbits.RC0 = 0;

```

```

TRISCBits.RC1 = 0;

ANSELC &= 0xF8;          // RC0,1,2 digital

// PORTD -> segments

TRISD  = 0x00;

ANSELD = 0x00;

// PORTA pins

TRISAbits.RA0 = 1;  ANSELABits.ANSA0 = 1;  // RA0 = AN0
(pot)

TRISAbits.RA1 = 1;  ANSELABits.ANSA1 = 0;  // RA1 = button
TRISAbits.RA2 = 0;  ANSELABits.ANSA2 = 0;  // RA2 = LED
TRISAbits.RA3 = 0;  ANSELABits.ANSA3 = 0;  // RA3 = servo
TRISAbits.RA4 = 0;          // RA4 = buzzer

// default outputs

LED      = 0;

ServoPWM = 0;

Buzzer   = 0;

PORTD    = 0xFF;          // all segments off (CA)

DIGIT1   = 0;

DIGIT2   = 0;

// ADC on AN0, Vref = Vdd/Vss, left-justified
ADCON0 = 0b00000001;  // AN0, ADC on
ADCON1 = 0x00;
ADCON2 = 0b00001001;  // ADFM=0, ACQT=2Tad, ADCS=Fosc/8
Delay10KTCYx(1);

```



```

    // Timer0: ~4 ms interrupt
    T0CON = 0b11010100;    // 8-bit, prescale 1:32, ON
    TMR0L = 256 - 125;
    INTCONbits.T0IF = 0;
    INTCONbits.T0IE = 1;
    INTCONbits.GIE = 1;

    // initial state
    counting = 0;
    set_val = 0;
    seconds = 0;
    Bin2Bcd(0, (char*)bcd);
}

// ===== read pot and scale 0..99
=====

static void ReadPot(void)
{
    ADCON0bits.GO = 1;
    while (ADCON0bits.NOT_DONE);    // wait for conversion
    adc_raw = ADRESH;                // left justification

    // scale 0..255 to 0..99 (UL: unsigned long)
    set_val = (unsigned char)((adc_raw * 100UL) / 256UL);
    if (set_val > 99) set_val = 99;
}

// ===== button logic =====

static void CheckButton(void)

```

```

{
    // if countdown is already running, ignore any presses
    if (counting)
        return;

    // simple press + release with small debounce
    if (BTN && (set_val != 0))
    {
        while (BTN);           // wait until user releases
        Delay10KTCYx(5);       // debounce delay

        // now we really start the countdown
        counting = 1;
        seconds  = set_val;
        LED      = 1;          // show that it is running
    }
}

// ===== servo pulse =====
static void ServoTask(void)
{
    if (counting && seconds > 0)
    {
        // full-speed in one direction (approx)
        ServoPWM = 1;          // high time ~2 ms
        Delay1KTCYx(2);        // 2 ms
        ServoPWM = 0;          // low time ~18 ms
        Delay1KTCYx(18);       // 18 ms -> total ~20 ms frame
    }
}

```

```

    else
    {
        // neutral / stop (close to 1.5 ms)
        ServoPWM = 1;
        Delay1KTCYx(1);    // 1 ms (you can try 1 or 2 to find
stop point)
        ServoPWM = 0;
        Delay1KTCYx(19);   // 19 ms -> ~20 ms
    }
}

// ===== buzzer =====

static void Beep(unsigned int LoopCount)
{
    unsigned int i;

    for (i = 0; i < 2 * LoopCount; i++)
    {
        Buzzer = ~Buzzer;
        Delay100TCYx(10);
    }
    Buzzer = 0;
}

// ===== Timer0 ISR =====

#pragma code ISR = 0x0008
#pragma interrupt ISR
void ISR(void)
{
    if (INTCONbits.T0IF)

```

```

{
    INTCONbits.T0IF = 0;

    TMR0L = 256 - 125;          // reload for next 4 ms
(approx)

    // choose which value to display
    if (!counting)
        Bin2Bcd(set_val, (char*)bcd);
    else
        Bin2Bcd(seconds, (char*)bcd);

    // turn both digits off before changing
    DIGIT1 = 0;
    DIGIT2 = 0;
    PORTD  = 0xFF;

    // enable one digit only
    if (qstates & 0x01) DIGIT1 = 1;    // ones digit
    if (qstates & 0x02) DIGIT2 = 1;    // tens digit

    // send pattern for the selected digit
    if (mux_idx == 0)
        PORTD = SS_CA[(unsigned char)bcd[2]]; // ones
    else
        PORTD = SS_CA[(unsigned char)bcd[1]]; // tens

    // next interrupt: other digit (flip mux_idx between 0
and 1)
    mux_idx ^= 0x01;

```

```

qstates <= 1;
if (qstates & 0x04) qstates = 0x01;

// build 1-second tick from 4-ms interrupts
if (--t0_ticks == 0)
{
    t0_ticks = 250;

    if (counting)
    {
        if (seconds > 0)
        {
            seconds--;

            // make noise at 3, 2, 1
            if (seconds > 0 && seconds <= 3)
                buzz_flag = 1;
        }
        else
        {
            // finished (0 reached)
            counting = 0;
            LED      = 0;
            set_val  = 0;      // next run starts from 0
again
        }
    }
}
}

```

}

4.2. Explanation Code

1. What the project does

The code turns the PIC18F45K22 into a **small microwave-style timer**:

- You turn a **potentiometer (RA0)** → it chooses a number from **00 to 99** seconds.
- You press and release a **button (RA1)** → it starts a **countdown**.
- The time is shown on a **2-digit 7-segment display** (PORTD + RC0/RC1).
- While counting:
 - **LED (RA2)** is ON.
 - **Servo (RA3)** is driven with a simple PWM signal.
 - **Buzzer (RA4)** beeps when only **3, 2, 1 seconds** are left.
- Everything is synchronized using **Timer0 interrupts**.

2. Pin usage

- **RA0** → Potentiometer (ADC input).
- **RA1** → Push button (start).
- **RA2** → LED.
- **RA3** → Servo signal.
- **RA4** → Buzzer.
- **PORTD** → Segments a–g (and dp) of the 7-segment display.
- **RC0** → Ones digit enable (right digit).
- **RC1** → Tens digit enable (left digit).

3. Main variables (what they store)

- `adc_raw` : value from the ADC (0..255).
- `set_val` : selected time from pot (0..99).
- `seconds` : remaining time during countdown.
- `counting` :
 - 0 → waiting / setting mode
 - 1 → countdown running

- bcd[3] : digits to show (hundreds, tens, ones) for the 7-segment.
- qstates & mux_idx : used for **multiplexing** the two digits (switching between tens and ones every interrupt).
- t0_ticks : counts how many Timer0 interrupts happened (used to build **1 second**).
- buzz_flag : a small flag to tell the main code “we should beep now”.

4. Program flow (main loop)

In main():

1. It calls Setup() once.
2. Then it stays forever in while(1).

Inside the loop:

- **If counting == 0 (not started yet):**
 - ReadPot() → read the pot, convert it to 0..99 → store in set_val.
 - CheckButton() → if the button is pressed and released and set_val != 0, start the countdown:
 - counting = 1;
 - seconds = set_val;
 - LED = 1;
- **If counting == 1 (countdown running):**
 - ServoTask() → generate pulses for the servo.
 - If buzz_flag == 1 → call Beep(200) to make a short sound, then clear buzz_flag.

So main is very simple:

setting mode → waiting for start

or

countdown mode → drive servo + buzzer when needed.

Time decrease (seconds--) does **not** happen in main; it happens inside the **interrupt**.

5. Setup (configuration)

Setup() configures everything:

- Sets RC0, RC1 as **outputs** for digit select.
- Sets PORTD as **output** for segments.
- Sets:

- RA0 = input, analog (pot, AN0),
 - RA1 = input, digital (button),
 - RA2, RA3, RA4 = outputs (LED, servo, buzzer).
- Clears outputs:
 - LED off,
 - ServoPWM low,
 - Buzzer low,
 - PORTD = 0xFF (all segments off),
 - both digits disabled.
- Configures **ADC**:
 - Uses AN0,
 - $V_{ref} = V_{dd} / V_{ss}$,
 - left-justified result (we read only ADRESH).
- Configures **Timer0**:
 - 8-bit mode,
 - prescaler 1:32,
 - preload TMR0L = 256 - 125 (this gives about 4 ms per interrupt),
 - enables Timer0 interrupt, and global interrupts.
- Initializes counting = 0; set_val = 0; seconds = 0; and converts 0 to BCD so the display shows 00.

6. Reading the potentiometer

ReadPot():

1. Starts an ADC conversion on AN0 (ADCON0bits.GO = 1;).
2. Waits until it finishes.
3. Reads ADRESH → adc_raw (0..255).
4. Converts to 0..99 by:
5. set_val = (unsigned char)((adc_raw * 100UL) / 256UL);
 - 100UL = 100 as unsigned long (for safe math).
 - Dividing by 256 scales 0..255 → almost 0..100.

6. If `set_val > 99`, we clamp it to 99.

So turning the pot slowly changes `set_val` between 0 and 99.

7. Button handling (start only once)

`CheckButton()`:

- First, if counting is already 1, it returns immediately → **button is ignored during countdown.**
- Then it checks if:
 - BTN is high (button pressed) **and** `set_val != 0` (we don't start if the value is 0).
- If yes:
 - It waits in `while (BTN)`; until the button is released.
 - Adds a small delay `Delay10KTCYx(5)`; for debouncing.
 - Then sets:
 - `counting = 1`;
 - `seconds = set_val`;
 - `LED = 1`; (to show that we are running).

So the timer **only starts after press + release** and only if the selected time is not zero.

8. Servo control

If `counting == 1` and `seconds > 0`:

The function generates a “run” pulse for the servo:

`ServoPWM = 1` for approximately 2 ms,

then `ServoPWM = 0` for the remainder of a 20 ms frame.

This pulse width instructs the continuous-rotation servo to rotate during the active countdown.

Else (when not counting or when `seconds == 0`):

The function outputs a “neutral” pulse, typically around 1–1.5 ms high followed by the rest of the 20 ms period low.

This places the servo in its stop/idle position.

9. Buzzer sound

`Beep(LoopCount)`:

- It flips (~) the buzzer pin many times inside a for loop.
- Each flip is separated by `Delay100TCYx(10)` → creates a simple squarewave sound.
- At the end, it forces `Buzzer = 0;` (silent).

`buzz_flag` is set inside the **ISR** when `seconds = 3, 2, 1`, and the **main** calls `Beep()` when it sees `buzz_flag == 1`.

10. Timer0 Interrupt (heart of the timing)

The `ISR()` is called every ~4 ms when Timer0 overflows:

1. Clears the Timer0 flag and reloads `TMR0L` for the next 4 ms.
2. Chooses what to show:
 - If not counting → convert `set_val` to BCD.
 - If counting → convert `seconds` to BCD.
3. Turns both digits off and clears segments.
4. Enables **only one digit** based on `qstates`:
 - If `bit0` is set → `DIGIT1` (ones) is enabled.
 - If `bit1` is set → `DIGIT2` (tens) is enabled.
5. Sends the correct pattern to `PORTD`:
 - For ones digit → `bcd[2]`.
 - For tens digit → `bcd[1]`.
6. Switches to the other digit for next time:
 - `mux_idx ^= 0x01;` → flips between 0 and 1.
 - `qstates <= 1;` and when it goes beyond `bit1`, it resets to `0x01`.

Because this happens every 4 ms, the two digits are **refreshed very fast**, so our eyes see them both ON continuously.

7. Builds the **1 second tick**:
 - Decrements `t0_ticks` every interrupt.
 - When `t0_ticks` reaches 0 → reload 250 → we treat this as 1 second passed.
 - If we are counting:
 - If `seconds > 0`:
 - `seconds--`.

- If seconds is now 3, 2 or 1 → sets `buzz_flag = 1`; (to beep in main).
- Else (`seconds == 0`):
 - Stop counting: `counting = 0`;
 - Turn LED off.
 - Reset `set_val` to 0.

4.3. Calculations

The time base is created with **Timer0** working:

- In **8-bit mode**.
- With **prescaler 1:32**.
- With **preload = 256 – 125** in TMR0L.

Each overflow generates an interrupt roughly every **4 ms** (depending on `Fosc`). In the interrupt service routine, a software counter `t0_ticks` is used:

```
unsigned char t0_ticks = 250; // 250 * 4 ms ≈ 1 s
```

Every Timer0 interrupt:

- `t0_ticks` is decremented.
- When `t0_ticks` reaches 0, it is reloaded to 250 and we consider that **1 second** has passed.

So the relationship is:

$$\begin{aligned}
 1 \text{ tick} &\approx 4 \text{ ms} \\
 250 \times 4 \text{ ms} &\approx 1000 \text{ ms} = 1 \text{ s}
 \end{aligned}$$

This is not a perfect real-time clock, but it is **good enough** for a lab timer and matches the project requirements.

The **countdown logic** in the ISR is:

- If `counting == 1` and `seconds > 0`, then `seconds--` every second.
- If `seconds` becomes 3, 2, or 1, set `buzz_flag = 1` so the main loop can call `Beep()`.
- When `seconds` reaches 0:
 - `counting = 0`;
 - LED is turned OFF.
 - The system goes back to setting mode.

5. Proteus

In Proteus, we checked:

- Changing the **potentiometer** value makes the 2-digit display update from **00** to **99**.
- The **countdown** does **not** start at power-up. It only starts after pressing and releasing the button.
- While counting down:
 - The **LED** on RA2 is ON.
 - The **servo** receives its PWM pulses and “runs” in simulation.
- When the remaining time is **3, 2, 1 seconds**, the **buzzer** toggles and we see the waveform on RA4.
- Pressing the button **during** the countdown is ignored, so the timer does not restart or glitch.

These tests confirmed that the logic of the program is correct before building the hardware.

Components:

- PIC18F45K22
- Resistors
- POT-HG (Potentiometer)
- Animated LED (Blue)
- 2 digits 7 segment CA
- Servo Motor PWM
- Push Button
- Sounder (Buzzer)

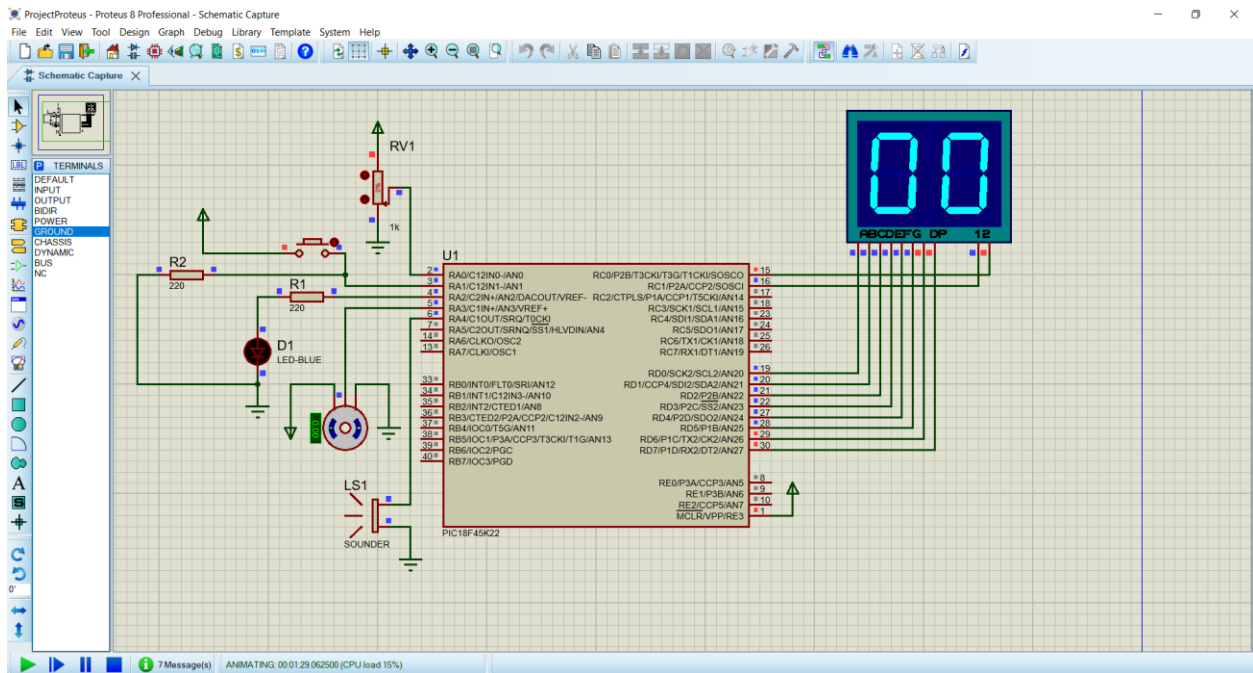


Figure 1: Proteus MinValue

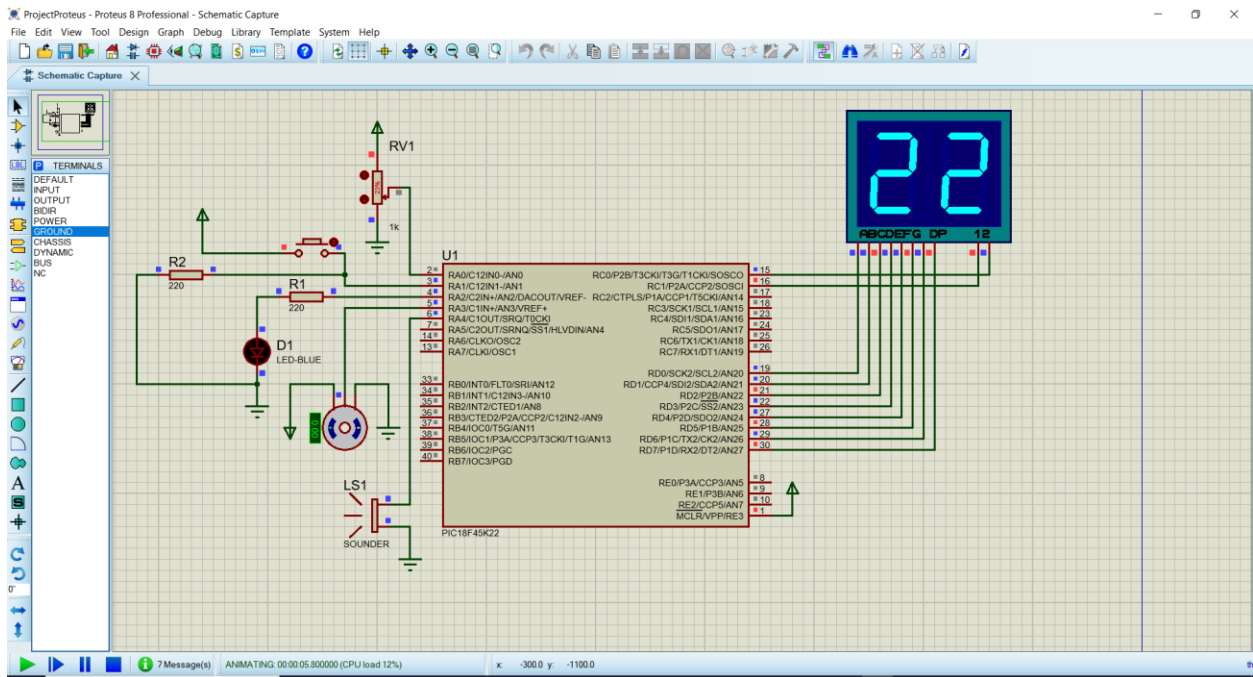


Figure 2: Proteus MidValue

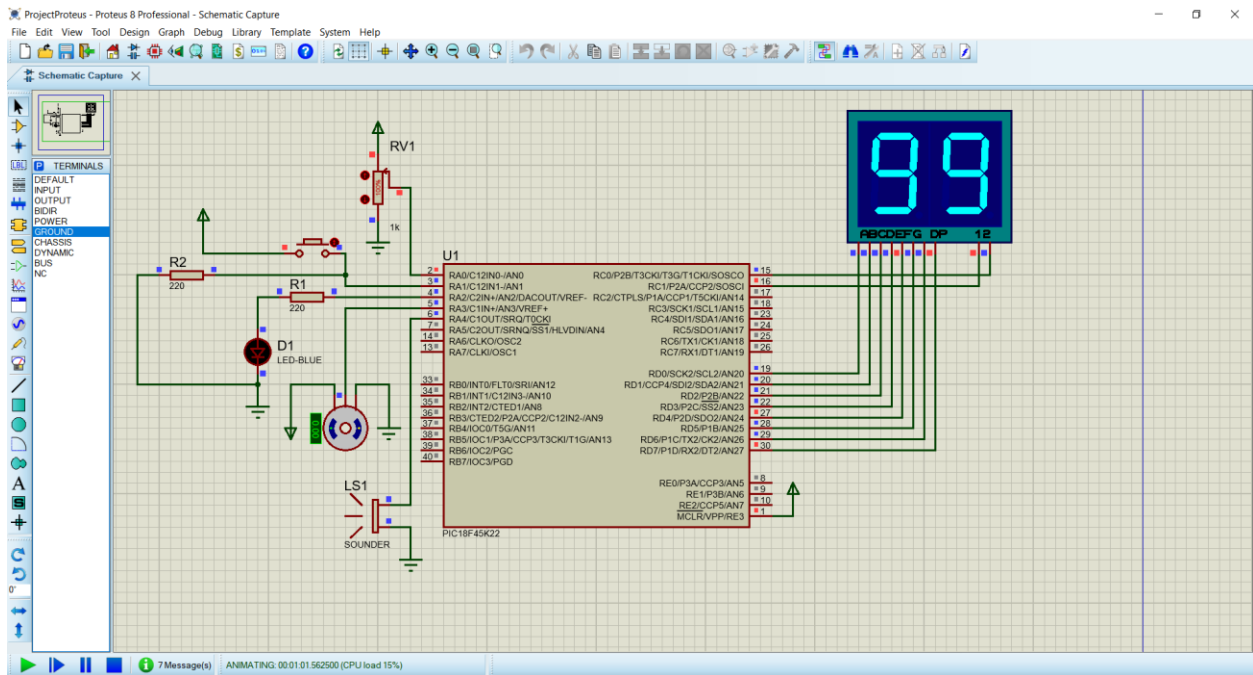


Figure 3: Proteus MaxValue

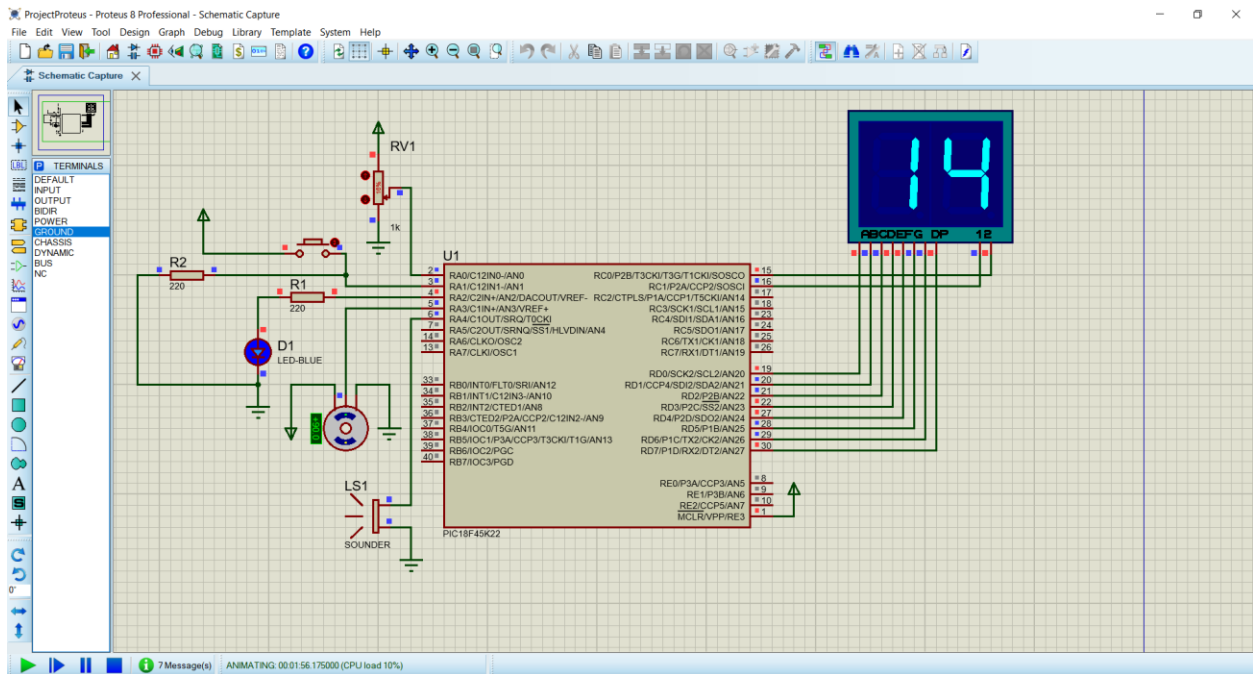


Figure 4: Proteus Running

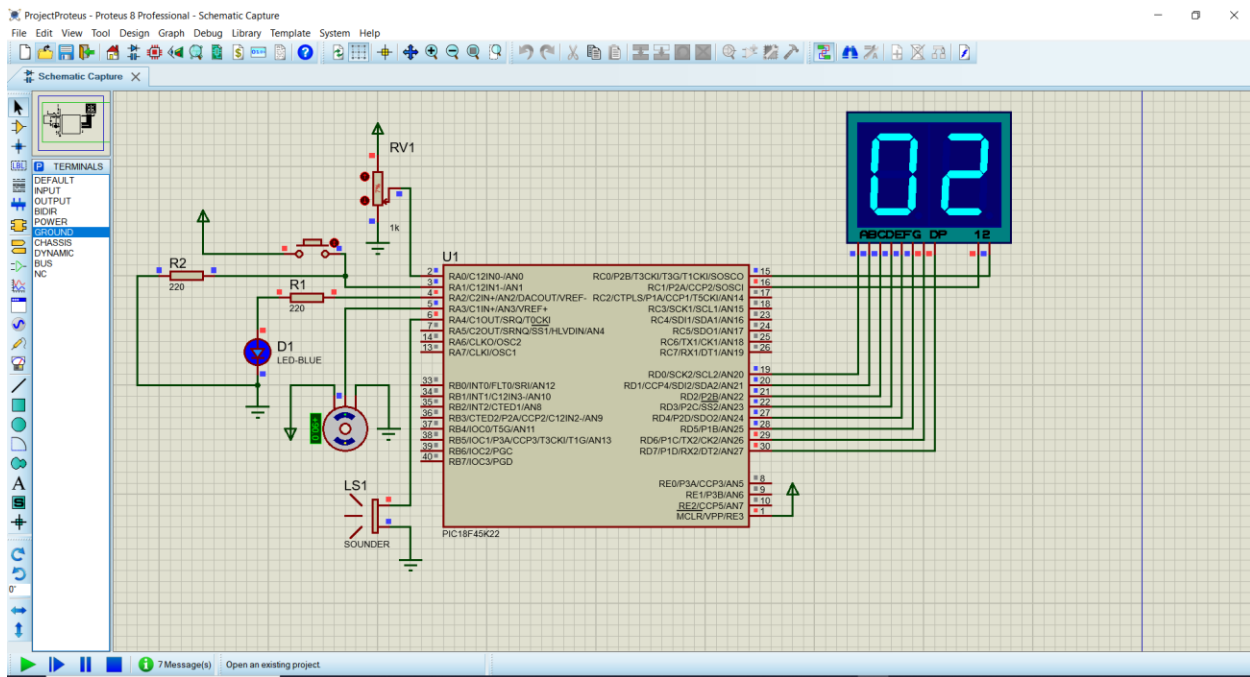


Figure 5: Proteus Running Buzzer

6. Hardware

6.1. Components

1. PIC18F45K22 microcontroller.
2. Dual common-anode 7-segment display (SN430502K) (2 digits).
3. FS5103R continuous-rotation servo motor.
4. Passive buzzer (5 V).
5. Push button (normally open) + resistor network.
6. LED and series resistor (e.g. 330 Ω).
7. Potentiometer (≈ 10 k Ω) for time selection.
8. 5 V linear voltage regulator (e.g. **7805**) + input/output capacitors.
9. External DC source (battery or adapter).
10. Breadboard and jumper wires.

6.2. Datasheets

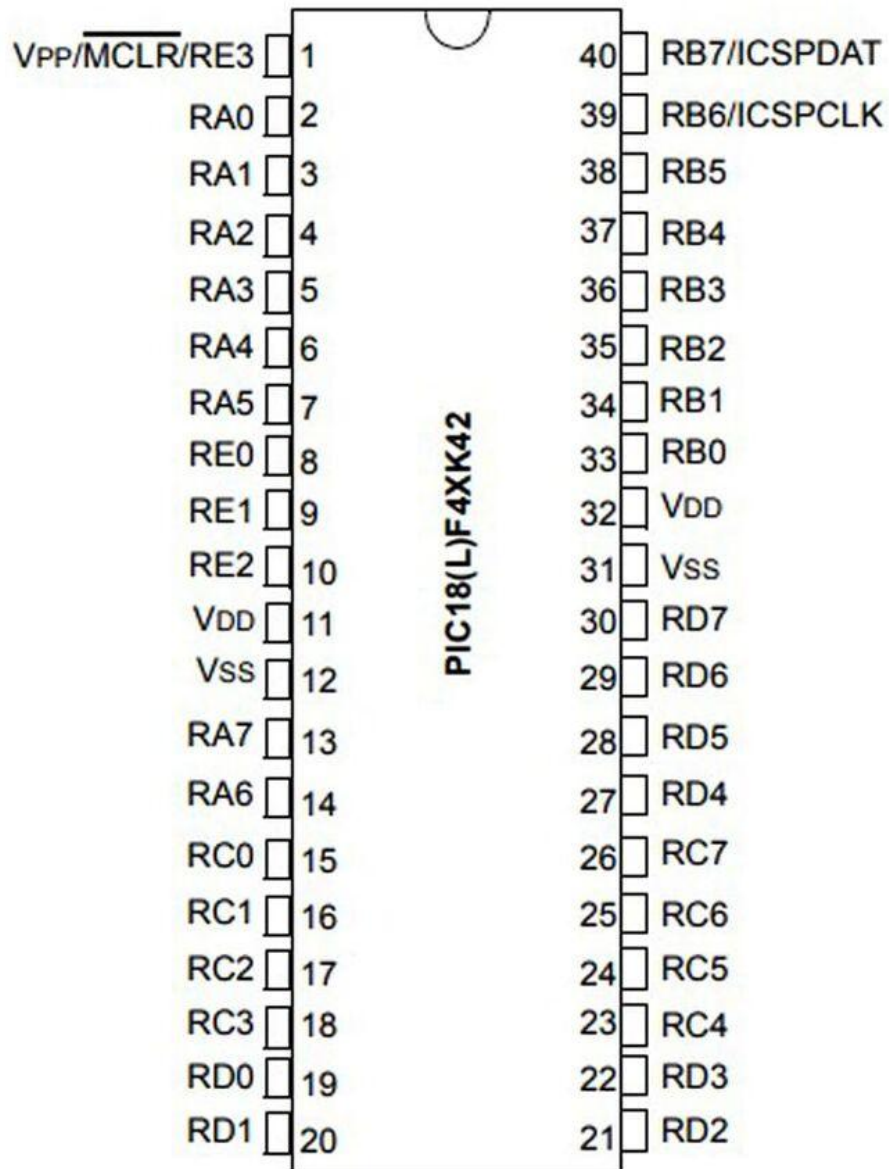


Figure 6: PIC18F45K22 Datasheet

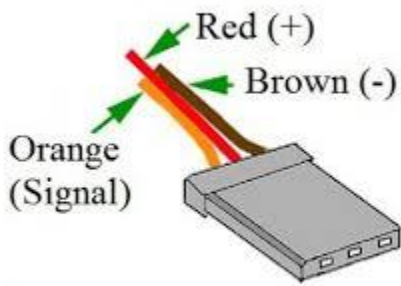


Figure 7: Servo Motor Datasheet

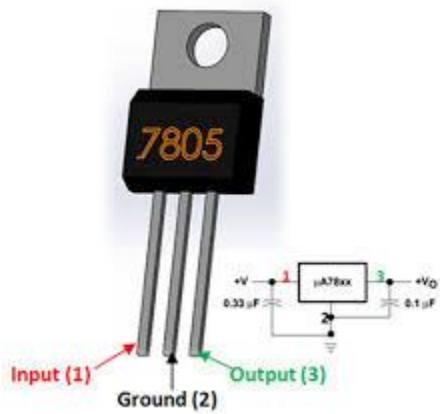


Figure 8: Voltage Regulator Datasheet

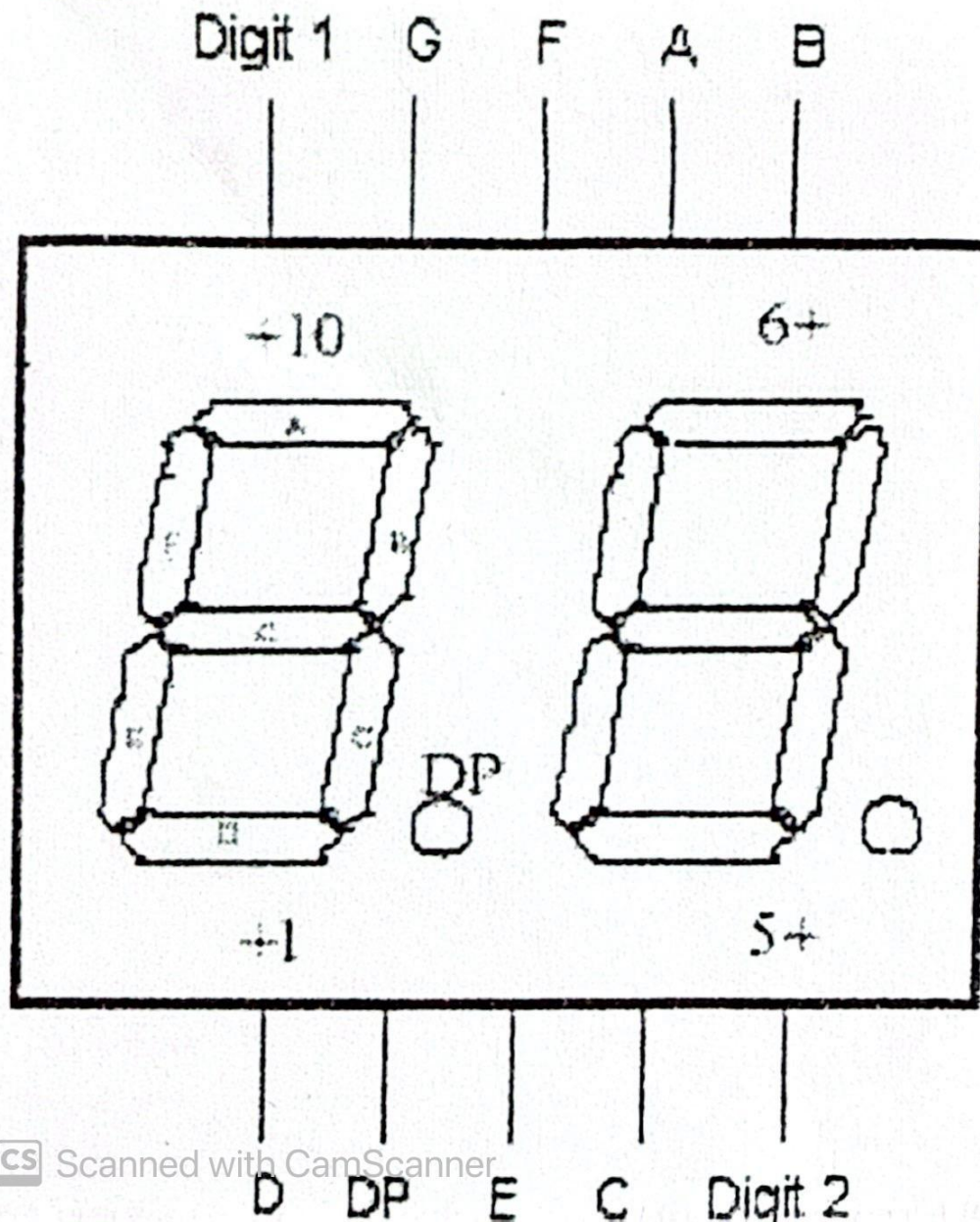


Figure 9: 7-Segment CA Datasheet

6.3. Power Supply and Voltage Regulator

- Use a **battery** or DC adapter (e.g. 9–12 V) as input to the **7805** regulator.
- Connect:
 - **IN** of the regulator to the positive of the battery (through a switch if you want).
 - **GND** of the regulator to battery negative.
 - **OUT** of the regulator is the **+5 V** rail for the whole circuit.
- Add decoupling capacitors:

- Typical: 0.33 μF at input, 0.1 μF + 10 μF at output (as per the datasheet).
- Connect:
 - PIC **VDD** pins to +5 V.
 - PIC **VSS** pins to GND.
 - 1Kohm resistor for master clear and to 5V
 - Servo, buzzer, LED, and 7-segment common anodes to +5 V.
 - All grounds **must be common** (PIC, servo, regulator, battery, etc.).

6.4. Microcontroller Wiring

- Oscillator and MCLR as in lab (or internal oscillator if configured like your previous labs).
- Connect **RA0** to the **middle pin** of the potentiometer. The two ends of the pot go to **+5 V** and **GND**.
- Connect **RA1** to one side of the push button; the other side goes to **+5 V**. Use a **pull-down** resistor from RA1 to GND so that RA1 reads 0 when the button is not pressed.
- Connect **RA2** to the LED (through a resistor) to GND.
- Connect **RA3** to the servo signal wire (usually the middle wire if the servo has three wires: power, ground, signal).
- Connect **RA4** to the positive terminal of the buzzer; the other terminal of the buzzer goes to GND (or use a small transistor if needed for current).

6.5. 7-Segment Display Wiring

- All segment pins (a, b, c, d, e, f, g, and optional dp) are connected to **PORTD**. The mapping must match the **SS_CA[]** array used in the code.
- The **common-anode** pins:
 - One digit's anode → **RC0** (DIGIT1).
 - Second digit's anode → **RC1** (DIGIT2).
- When **RC0 = 1** and **RC1 = 0**, the **ones** digit is active.
When **RC0 = 0** and **RC1 = 1**, the **tens** digit is active.
- Small series resistors (e.g. 220–330 Ω) can be added for each segment to limit current.
- Add pnp resistor to each digit and a resistor to each.

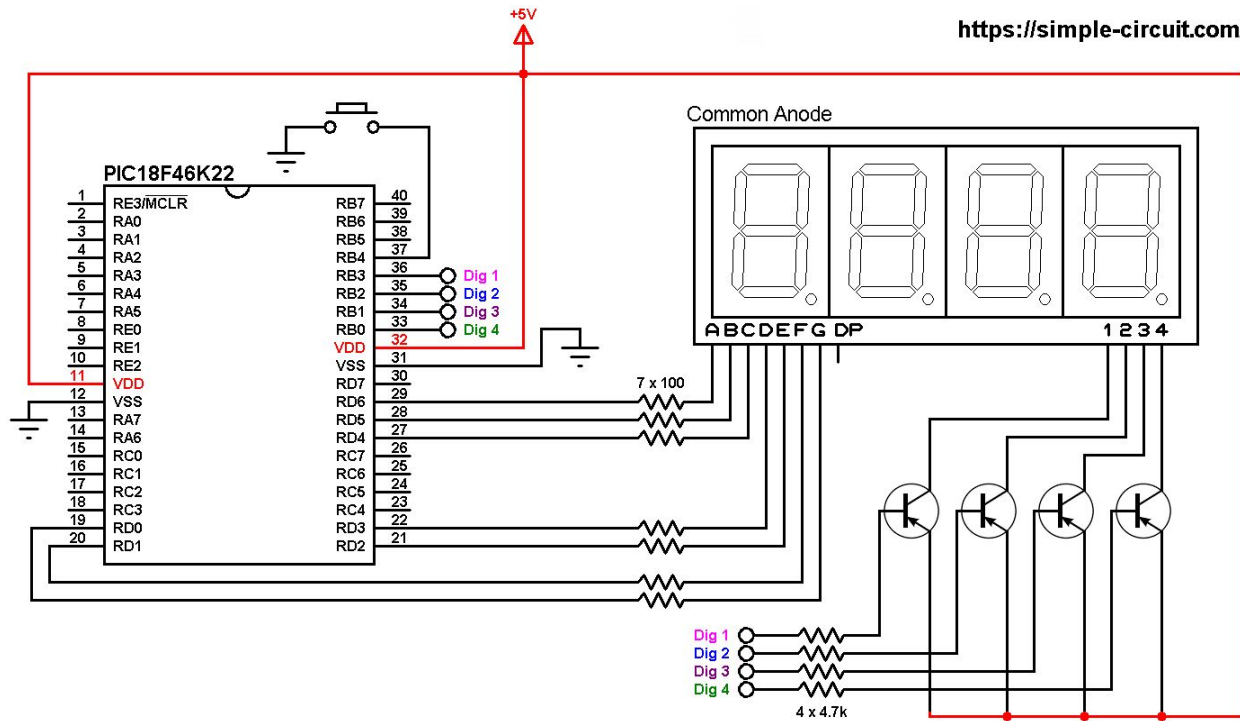


Figure 10 Circuitry of 7 segment display

6.6. Programming with PICKit (Steps)

1. Plug chip into the pickit3

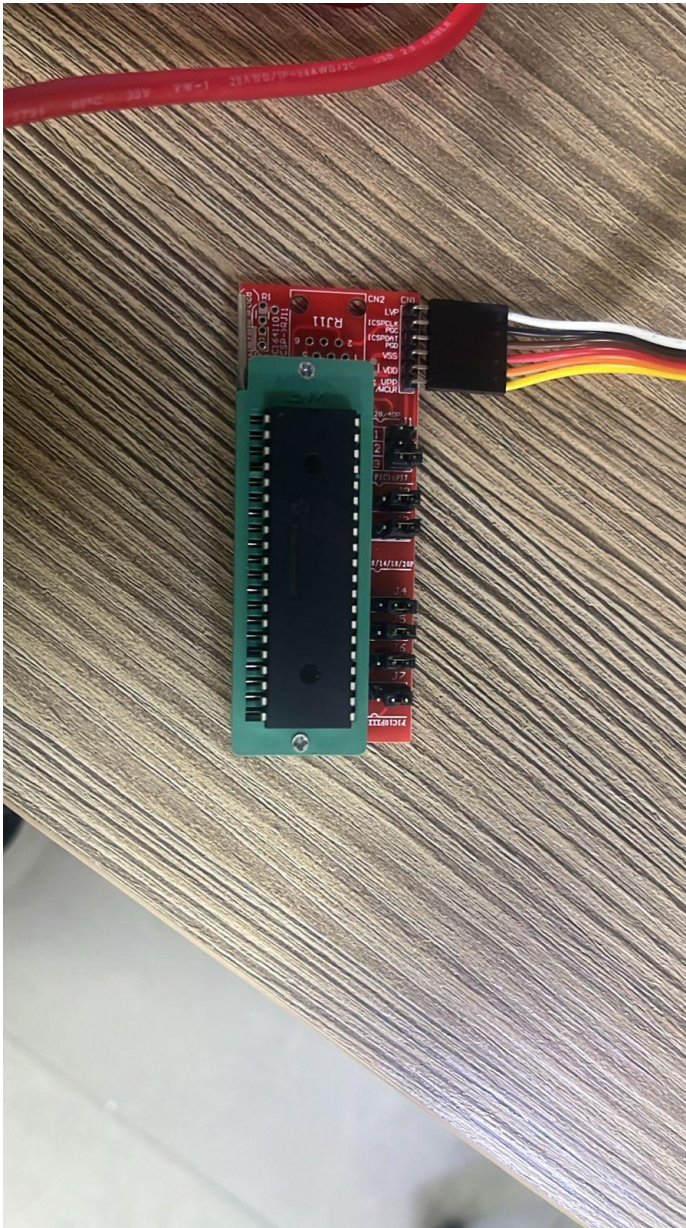


Figure 11 hardware-1

2. We add in the code to enable the internal oscillator and change this pragma in the pic18f45k22.h library (we commented it)

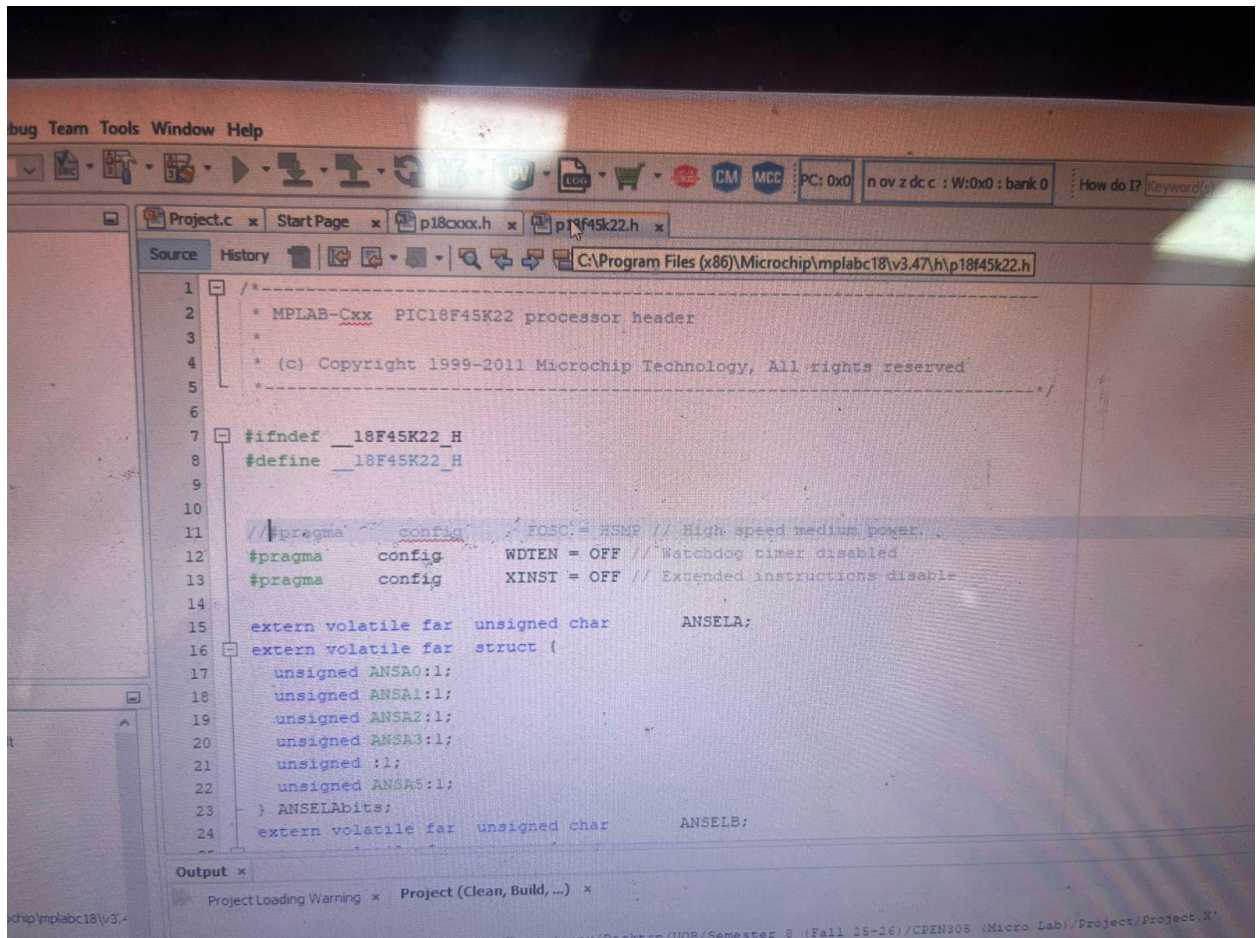


Figure 12 hardware-2

3. We plug the pickit to the pc and go in mplab to production
4. Then set project configuration to customize

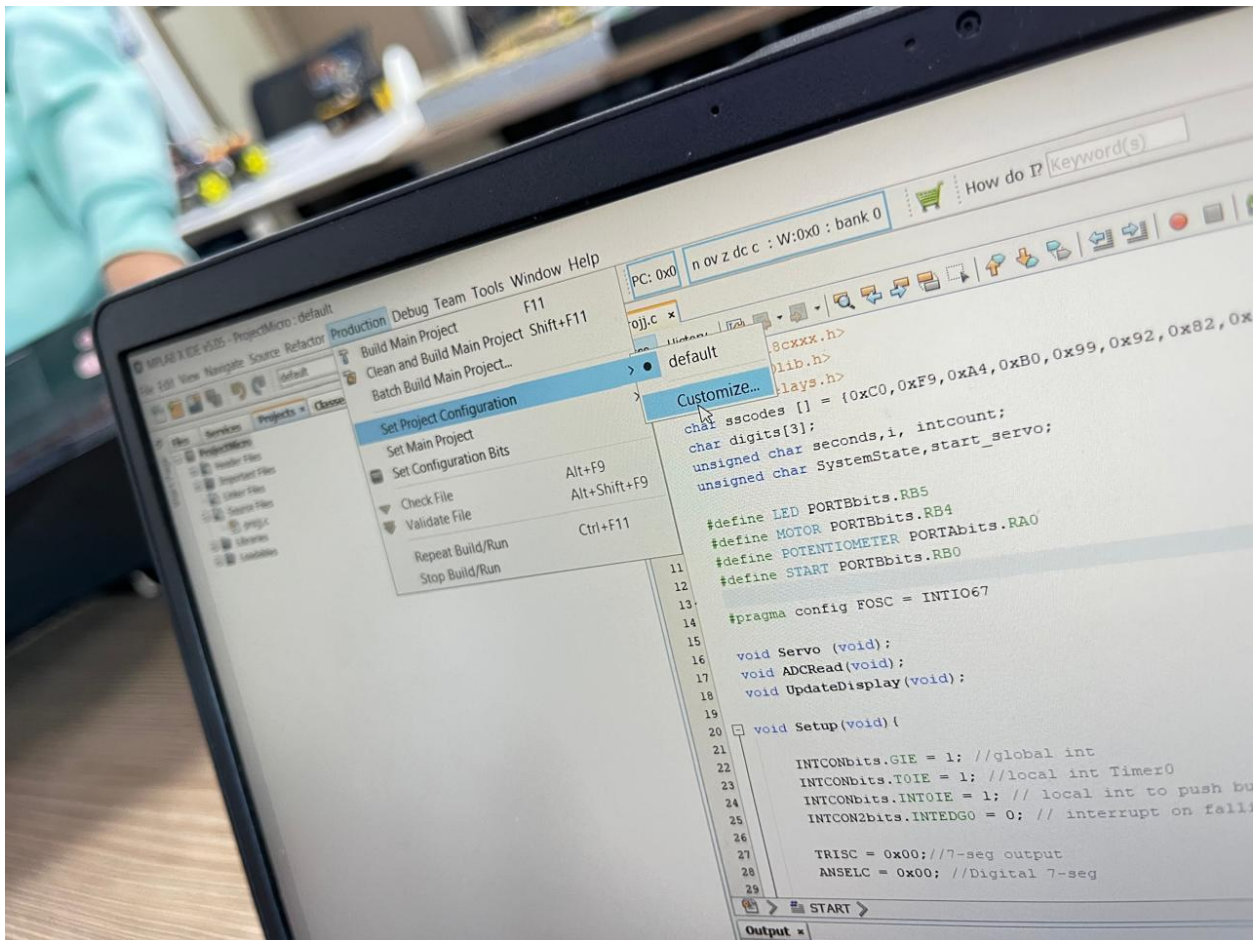


Figure 13 hardware-3

5. pickit3 option category to power

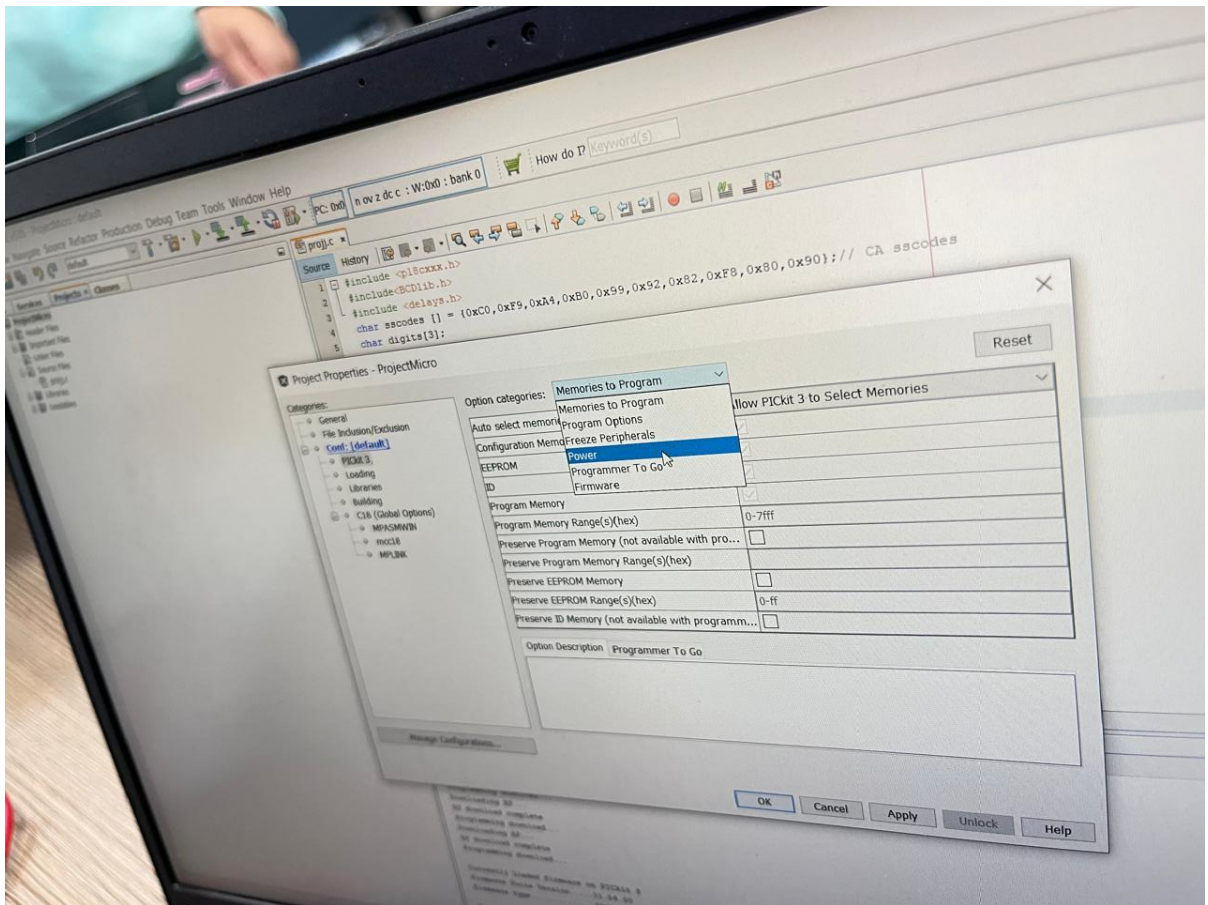


Figure 14 hardware-4

6. Apply

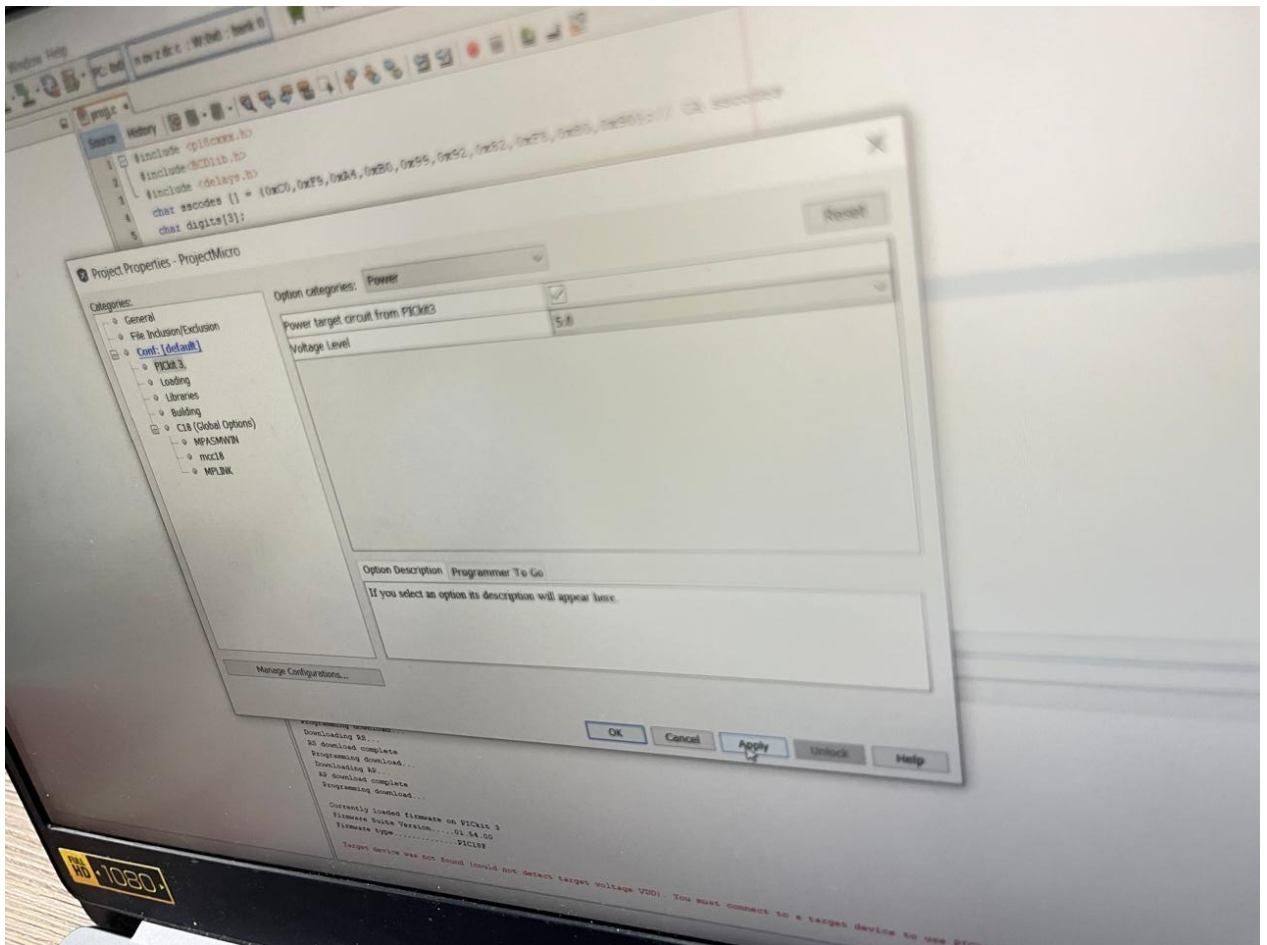


Figure 15 hardware-5

7. We press run and wait for complete

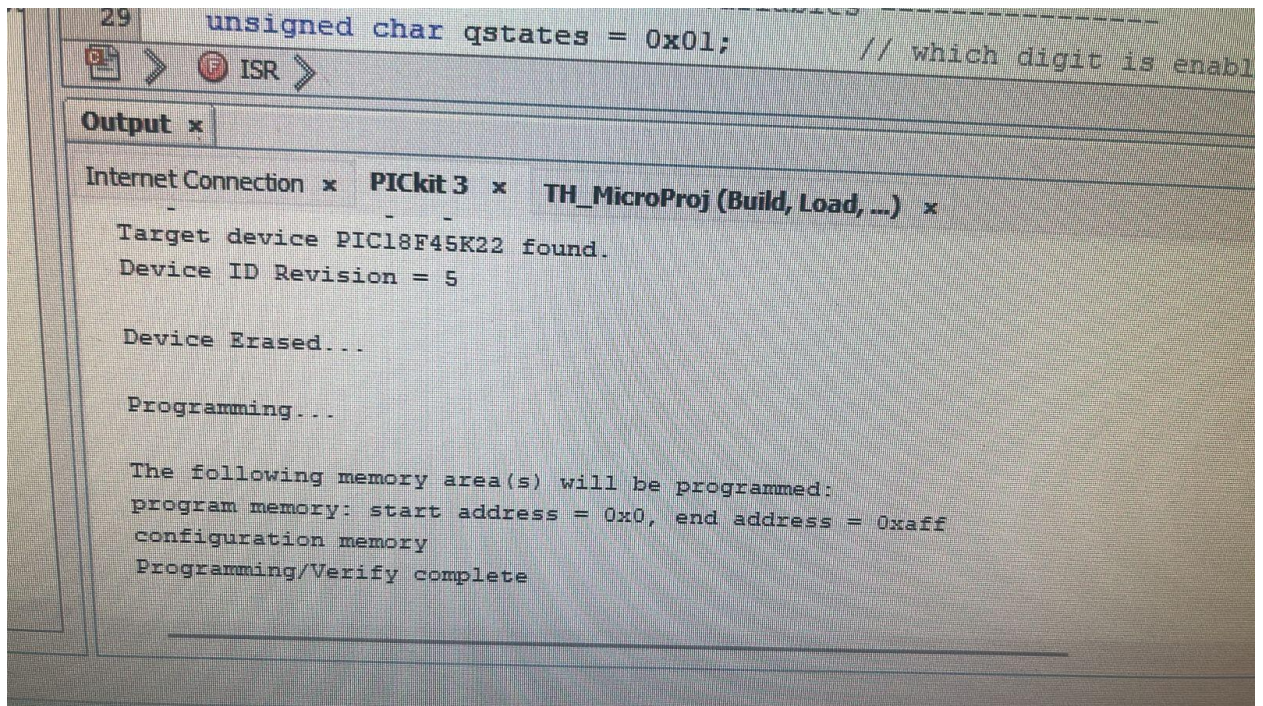


Figure 16 hardware-6

8. We press download button and wait to complete

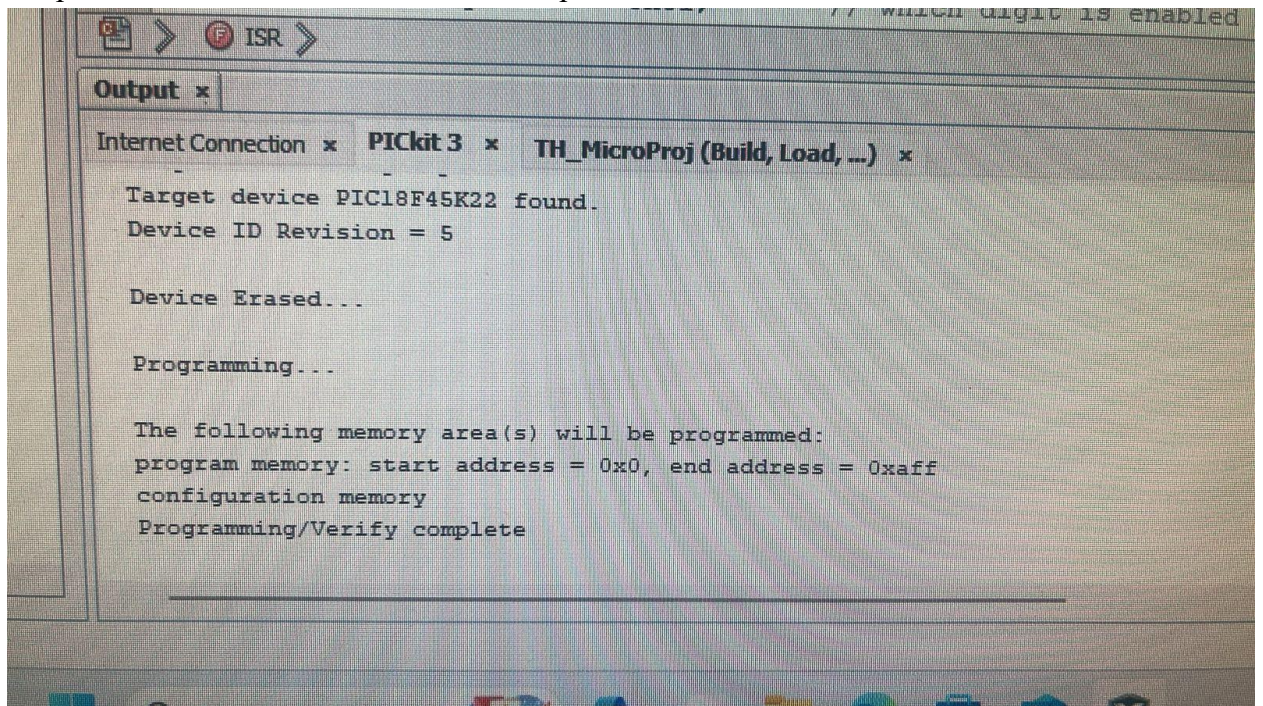


Figure 17 hardware-7

9. We re-plug the programed pic18 into the circuitry we did and voila

6.7. Screenshots

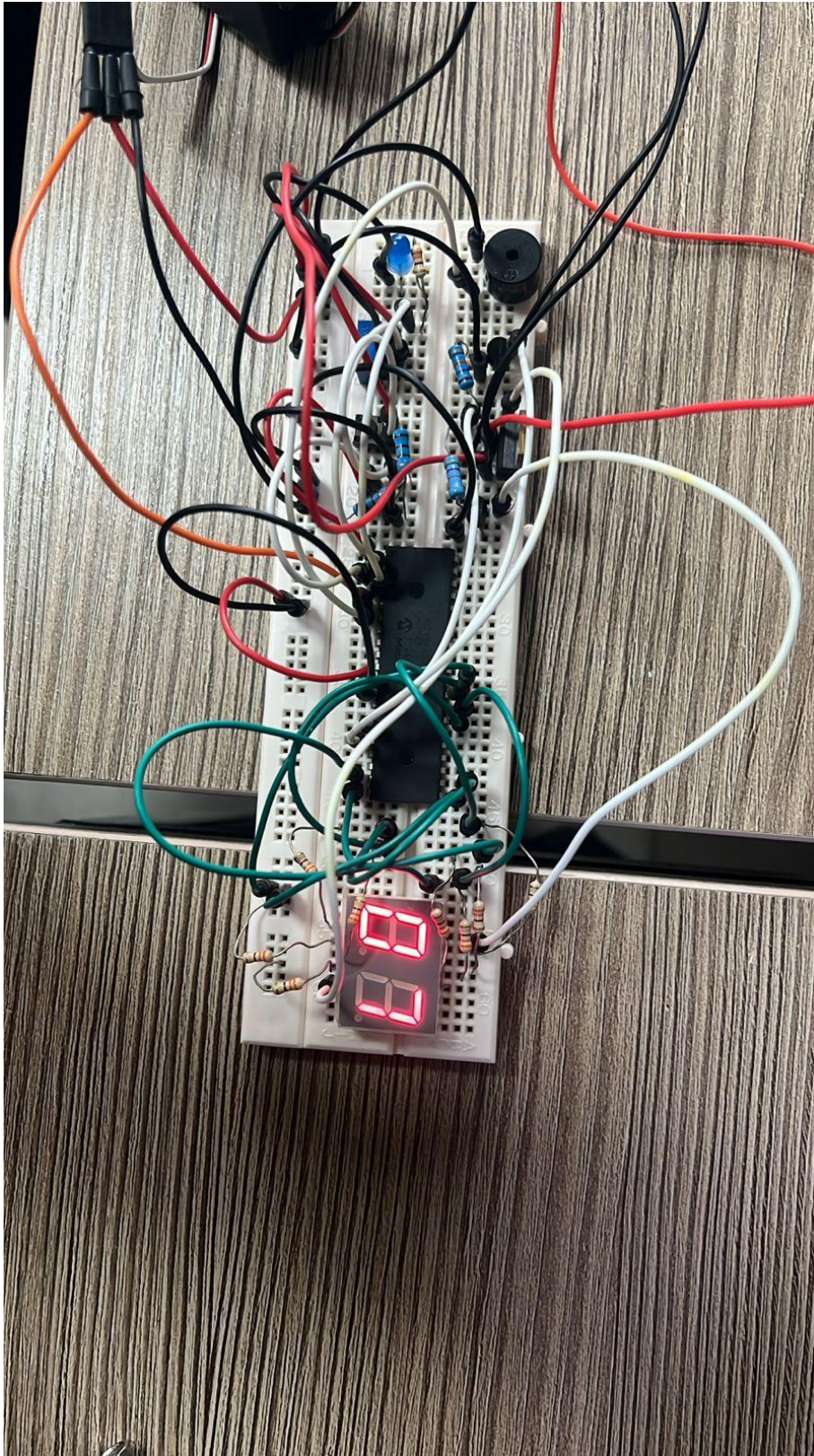


Figure 18 final hardware

7. Future Work

Some ideas to extend this project:

- **Keypad input instead of potentiometer**
Replace the pot with a **numeric keypad** so the user can enter exact times (e.g. 30 s, 90 s).
- **LCD user interface**
Add a **16×2 LCD** to display text messages like “Set Time”, “Running”, “Done”, and show time in **MM:SS** format.
- **More accurate timing**
Use a **crystal oscillator** and configure Timer modules to achieve very accurate seconds, closer to a real microwave.
- **Multiple power-cut modes**
Extend the EEPROM logic to store more states (last time used, last mode, maybe pause/resume state) for better behavior after power loss.

8. Conclusion

In this project, we designed and tested a **microwave timer system** based on the **PIC18F45K22**. The system combines:

- ADC reading from a potentiometer,
- multiplexed dual 7-segment display,
- debounced push button control,
- LED and servo control during countdown,
- and a buzzer alarm in the last three seconds.

Proteus simulations helped verify the behavior before hardware implementation. The final design respects the constraints and techniques used throughout the **CPEN305 Microprocessor Lab** and can be used as a base for more advanced embedded timing applications.